Chapter 4 | Programming Fundamentals & Problem Solving

4.1 Problem Solving and Algorithms

Algorithm: a **step-by-step solution of a problem**, written before any code. It is **not dependent on any programming language**, and because its steps are precise, an **algorithmic solution's correctness can be checked very easily** — algorithms are easy, not difficult, to debug. Every algorithm must satisfy five properties:

(1) Input — zero or more quantities supplied.

(2) Output — **at least one quantity must be produced** (the claim that some algorithms may produce no output is the false property).

(3) Definiteness — **each instruction is clear and unambiguous** (the named characteristic).

(4) Finiteness — it terminates after a finite number of instructions, in all cases.

(5) Effectiveness — every step is basic enough to be carried out.

Efficiency: measured in terms of **time and space**. **Space complexity = the amount of memory required to execute the algorithm**; time complexity = the running time.

Randomised algorithms: algorithms in which some decisions depend on a randomiser's output. **Las Vegas algorithms always produce the same correct output**, but their **execution time DOES depend on the randomiser** (the tested incorrect claim says it doesn't). **Monte Carlo algorithms always run in polynomial time but may return erroneous answers** — fixed time, possible error; Las Vegas is the mirror: guaranteed answer, variable time.

(1) DFS explores as far as possible along each branch before backtracking; BFS visits all vertices at the current depth before the next level — both can traverse trees and graphs (stack vs queue).

(2) Top-down design order: Analyse the problem → write the main module → write the remaining modules → connect the modules → revise and resequence (I → III → V → IV → II).

(3) Subprograms include BOTH functions and subroutines; **fork() and join() together make a complete process subroutine** (create, then wait). In computational-thinking terms, solving a Sudoku efficiently rests most on **pattern recognition**.

Note: Two strays keyed in this band: the RSA computation — p = 7, q = 11 → φ = 60; e must satisfy 37e ≡ 1 (mod 60) → **e = 13** (full RSA treatment is with Networks); and a Best Fit recap — its cost is **increased search time for free blocks** (Chapter 3 territory).

Exam Points — 4.1

- ✦ Algorithm = step-by-step, language-independent, easy to debug and verify. Properties: input, output (≥1 compulsory), definiteness (= unambiguous instructions), finiteness, effectiveness.
- ✦ Efficiency = time + space; space complexity = memory needed. Las Vegas = always correct, time varies with the randomiser; Monte Carlo = polynomial time, may err.
- ✦ DFS = deep then backtrack; BFS = level by level. Top-down: I→III→V→IV→II. Functions AND subroutines are subprograms; fork() + join(); Sudoku → pattern recognition; RSA worked e = 13.

4.2 Flowcharts and Pseudocode

Flowchart: the **graphical (pictorial) representation of an algorithm or program** — asked in both wordings. Its symbols:

Symbol	Meaning
Oval	Start / End (terminator)
Parallelogram	Input / Output
Rectangle	**Process** — the processing phase
Diamond	**Decision** — a branch; a diamond-based branch in a flowchart is the decision structure
Circle	**Connector** — links one part of the flowchart with another

Control constructs: sequence — each statement is executed **once** and **no statement is repeated**, in written order (statements I and II true; "any order" false); **decision/selection** (if–else, case); **loop/iteration** (repetition).

Pseudocode: algorithm steps written in code-like plain language — of algorithm, flowchart and pseudocode, **pseudocode is the easiest to transform into a computer program** (asked twice).

Exam Points — 4.2

- ✦ Flowchart = graphical/pictorial form of an algorithm/program. Oval = start/end; parallelogram = I/O; rectangle = process; diamond = decision; circle = connector.
- ✦ Sequence structure: every statement once, none repeated, in order. Pseudocode → easiest to convert into a program.

4.3 Language Translators — Compiler, Interpreter, Assembler

A program written in a high-level language must be translated before the CPU can run it. The standard pipeline:

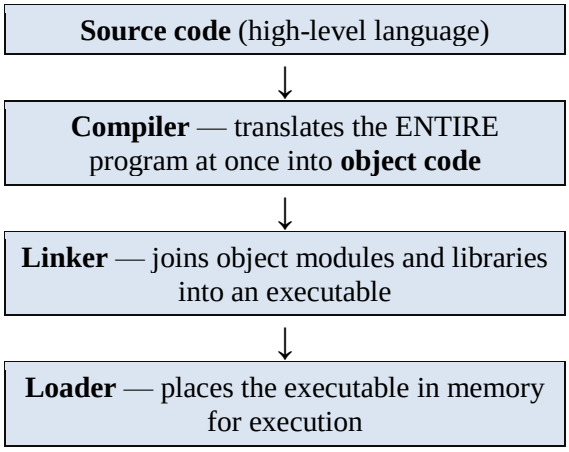


Fig. 4.1 — The translation pipeline. The compiler is not involved in the program's execution.

Translator	Asked facts
Compiler	**System software package converting a high-level language program to machine language**; **translates source code into object (target) code**; reads the **entire** source program (never line by line — that claim is false); reports errors; **not involved in execution**
Interpreter	**Translates and executes each statement ONE AT A TIME**; **BASIC is an interactive language**; **Perl is typically interpreted, C is compiled** (Python interpreted too)

Assembler	**Translates an assembly-language program into its machine-language equivalent**; assembly languages are **machine dependent**; a **pseudo-instruction (directive) is given to the assembler**

(1) **Machine language**: made of 1s and 0s, machine dependent, difficult to program — the FALSE claim is that it is "easier to modify."

(2) **JIT (Just-In-Time) compiling** = compiling Intermediate Language programs to actual machine language on the target platform **prior to execution** ("First-In-Time" is invented).

(3) The **command interpreter (shell)** exists to **get and execute the next user-specified command**.

(4) **Compiler phases** in order: lexical analysis (tokens) → syntax analysis (grammar) → **semantic analysis — gathers TYPE information and saves it in the syntax tree or SYMBOL TABLE** for intermediate code generation → intermediate code → optimisation → code generation.

Macros and Assembler Directives

(1) A **macro** is a **single-line abbreviation for a group of instructions**; its **name is used as an operation mnemonic**; it is invoked **by simply writing its name with any required parameters** (no CALL, no JMP); every call is **replaced by the macro body** at assembly time. The FALSE statement: "the macro definition also appears in the expanded source code" — the definition is consumed, only expansions appear.

(2) **Assembler directives**: DB, ORG, END — they guide assembly. **MOV is an executable instruction, not a directive.**

Exam Points — 4.3

- ✦ Compiler: HLL → machine, source → object, whole program at once, no role in execution. Interpreter: statement at a time; BASIC interactive; Perl interpreted, C compiled. Assembler: assembly → machine; machine dependent.
- ✦ Machine language is NOT easy to modify. JIT = IL → native before execution. Command interpreter gets and executes the next command. Semantic analyzer stores type information in the symbol table.
- ✦ Macro: name = mnemonic, invoked by name + parameters, expanded inline, definition absent from expansion. Directives = DB/ORG/END; MOV is an instruction. Pseudo-instruction → assembler.

4.4 Language Generations and Software Types

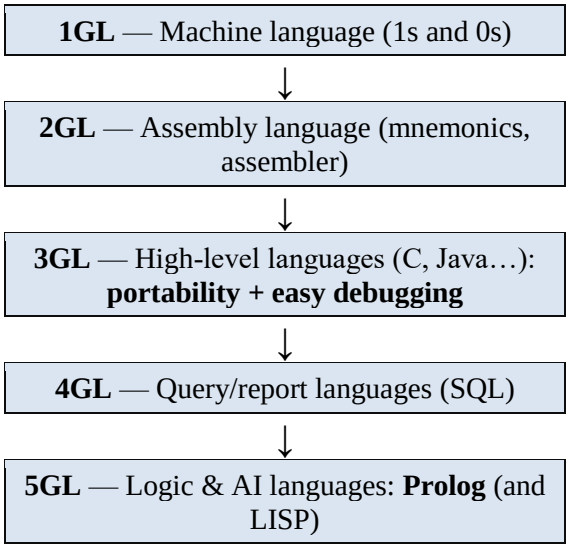


Fig. 4.2 — The five language generations. The asked 5GL example is Prolog — not Java, COBOL or Python.

System software is typically written in **low-level language** and **plays a vital role in the effective functioning of the system** — the false claim is that it is written in a high-level language; **application software is written in high-level languages**. A **software package** = **a set of computer programs with the necessary documentation**. For organising notes to interpret information, the valid method among those asked is the **flow chart**.

Exam Points — 4.4

- ✦ Prolog = fifth-generation language. High-level languages give portability + easy debugging (both). Perl interpreted; C compiled.
- ✦ System software: low-level, vital to the system; application software: high-level. Software package = programs + documentation. Note organisation → flow chart.

4.5 Variables, Data Types, Scope and Recursion

Variable: a symbolic name given to a memory location — unlike a constant or literal, its value can change. A variable must have **a storage location AND a data type** (a sign bit only for some numeric types — the safe answer is all of the essentials).

Data type: the specification of legal values for variables of that type — the set of permitted values plus the operations allowed on them (int, float, char).

(1) **Scope of a variable declared inside a function: only inside the { } block** — local scope, automatic lifetime.

(2) An **external (global) variable**: **globally accessible by all functions, declared with extern inside other functions, and initialised to 0 by default** — all three true.

(3) In a **recursive** call, all **automatic variables are initialised afresh during each call** (each call gets a new activation record on the stack — nothing carries over; use static to persist). A **recursive procedure calls itself again and again until the base condition** is reached.

(4) In C++, a **namespace defines a SCOPE** for the identifiers used in a program (std::cout).

Note: Stray filed here: the SSID item — an SSID is a human-readable string of at most **32 bytes** (not 2000 — statement i false) and **acts as the single shared identifier between access points and clients** (true). Full wireless treatment is in Chapter 10.

Exam Points — 4.5

- ✦ Variable = symbolic name of a memory location; needs storage + data type. Data type = specification of legal values.
- ✦ Function-local scope = inside the braces. External variable: global + extern + default 0 (all). Recursion: automatic variables re-initialised each call; runs till the base condition. Namespace = scope.

4.6 Errors, Debugging and Testing

Error stage	What it is
Syntax error	Grammar violation — caught at **compile time**
Semantic / logic error	Legal code, wrong meaning — wrong output, no crash
Run-time error	Arises during execution; **the condition it causes is an EXCEPTION** (divide by zero)

Debugging approaches: brute force — the program is loaded with print statements to display intermediate values, hoping one reveals the faulty statement (the asked method); backtracking — trace backward from the failure; cause elimination — hypothesise and test causes; program slicing — isolate the statements affecting a value. High-level languages make debugging easy — one of their two tested features.

Exam Points — 4.6

✦ Run-time error → exception (syntax = compile time; semantic = wrong output).
✦ Print-statement debugging = brute force method.

Quick Revision — 20 points

1. Algorithm = step-by-step, language-independent, easily verified and debugged. Properties: input, output (at least one), definiteness (unambiguous), finiteness, effectiveness.
2. Efficiency = time + space; space complexity = memory required.
3. Las Vegas: always correct, time depends on the randomiser. Monte Carlo: polynomial time, may err.
4. DFS: deep along a branch, then backtrack (stack). BFS: level by level (queue).
5. Top-down design: analyse → main module → remaining modules → connect → revise (I III V IV II). Subprograms = functions + subroutines. fork() + join() = complete process subroutine.
6. Sudoku via computational thinking → pattern recognition. RSA worked: $\varphi = 60$, $e = 13$.
7. Flowchart = graphical/pictorial algorithm. Oval start/end; parallelogram I/O; rectangle process; diamond decision; circle connector.
8. Sequence: each statement once, none repeated, in order. Diamond branch = decision structure. Pseudocode = easiest to turn into a program.
9. Pipeline: source → compiler → object code → linker → loader.
10. Compiler: whole HLL program → machine/object code at once; system software package; uninvolved in execution. Interpreter: one statement at a time; BASIC interactive.
11. Assembler: assembly → machine equivalent; assembly is machine dependent; pseudo-instructions are for the assembler.
12. Machine language: 1s/0s, machine dependent, hard to program, NOT easy to modify.
13. JIT = intermediate language compiled to native code before execution. Command interpreter = get and execute the next user command.
14. Compiler phases: lexical → syntax → semantic (type info → symbol table) → intermediate → optimise → generate.
15. Macro: one-line abbreviation, name as mnemonic, invoked by name + parameters, calls replaced inline, definition never in expansion. Directives DB/ORG/END; MOV is an instruction.
16. Generations: machine → assembly → HLL (portable, easy debug) → 4GL → 5GL (Prolog, LISP). Perl interpreted; C compiled.
17. System software = low-level + vital; application = high-level. Software package = programs + documentation.
18. Variable = named memory location (needs storage + type). Data type = legal values + operations. Local scope = the {} block; external variable = global, extern, default 0; namespace = scope.
19. Recursion: calls itself till the base condition; automatic variables re-created and initialised on every call (stack frames).
20. Errors: syntax = compile time; run-time → exception. Brute-force debugging = print statements everywhere.

DSSSB – TGT COMPUTER SCIENCE (PRACTICE KING) BOOK
Call/Message: 7011121290 Available @ Amazon: https://amzn.in/d/07E04o9B